

Otulia Web: Deep Dive - Chapter 7: Search Engine Optimization (SEO)

7.1 Introduction to Modern SEO

For a luxury marketplace like Otulia, "Discoverability" is everything. We don't just want to be on Google; we want to dominate search results for specific luxury brands and models. This chapter explores the multi-layered SEO strategy we've implemented to ensure Otulia is fully optimized for all modern search engines.

7.2 Dynamic Metadata Architecture (SEO.jsx)

In a Single Page Application (SPA), the browser title and meta tags don't change automatically when you click a link. We use `react-helmet-async` to solve this.

7.2.1 The SEO Component Breakdown

```
export default function SEO({ title, description, image, url }) {  
  // 1. Consistent Branding: Every title ends with " | Otulia"  
  const seoTitle = title ? `${title} | Otulia` : 'Otulia - Luxury Assets';  
  
  return (  
    <Helmet>  
      {/* 2. Standard Meta: Essential for Google Search Results */}  
      <title>{seoTitle}</title>  
      <meta name="description" content={description} />  
  
      {/* 3. Open Graph (OG): Makes links look premium on Facebook & iMessage */}  
      <meta property="og:title" content={seoTitle} />  
      <meta property="og:image" content={image} />  
  
      {/* 4. Twitter Cards: Optimized for high-res image sharing on X */}  
      <meta name="twitter:card" content="summary_large_image" />  
      <meta name="twitter:title" content={seoTitle} />  
    </Helmet>  
  );  
}
```

7.3 Google Sitelinks & Structured Data

Have you ever seen a Google result with sub-links like "Login," "Shop," and "About"? We "request" these from Google using **JSON-LD Structured Data**.

7.3.1 Site Navigation Schema

We inject a script that explicitly tells Google's bot: *"These are the 6 most important pages on my site."*

```
const navSchema = {  
  "@context": "https://schema.org",
```

```

"@type": "ItemList",
"itemListElement": [
  { "@type": "SiteNavigationElement", "position": 1, "name": "Shop", "url":
"https://otulia.com/shop" },
  { "@type": "SiteNavigationElement", "position": 2, "name": "Rent", "url":
"https://otulia.com/rent" },
  // ...
]
};

```

7.4 The Dynamic Sitemap Engine (sitemap.routes.js)

Google needs a "Map" to find all your listings. Instead of a static file, we generate this XML map in real-time from the database.

7.4.1 Sitemap Generation Logic

```

router.get('/sitemap.xml', async (req, res) => {
  // 1. Fetch live "Active" listings only
  const [cars, yachts, estates] = await Promise.all([
    CarAsset.find({ status: 'Active' }),
    YachtAsset.find({ status: 'Active' }),
    // ...
  ]);

  // 2. Build XML Header
  let xml = `<?xml version="1.0" encoding="UTF-8"?><urlset xmlns="...">`;

  // 3. Dynamic Link Injection: Every new car listed automatically gets a URL here
  cars.forEach(asset => {
    xml += `<url>
      <loc>https://otulia.com/asset/car/${asset._id}</loc>
      <lastmod>${asset.updatedAt.toISOString()}</lastmod>
      <priority>0.9</priority> { /* Tells Google this is a high-value page */}
    </url>`;
  });

  res.header('Content-Type', 'application/xml');
  res.send(xml);
});

```

7.5 Crawl Control (robots.txt)

We guide search engine bots away from "noise" (like the Admin panel or User Profile) so they spend their "Crawl Budget" on your actual luxury listings.

```

User-agent: *
Disallow: /admin/
Disallow: /profile

```

```
Disallow: /cart
```

```
Sitemap: https://otulia.com/sitemap.xml
```

7.6 Implementation in the UI

To make this work, the `<SEO />` component is dropped into the top of every page.

7.6.1 Usage on the Asset Page

```
// In Car_Section.jsx
if (!info) return <Loading />;

return (
  <>
    <SEO
      title={`${info.brand} ${info.title}`} // e.g., "Ferrari SF90 Stradale | Otulia"
      description={info.description}
      image={info.images[0]}
    />
    <CarContent />
  </>
);
```

By following this architecture, Otulia ensures that every luxury asset—no matter how new—is instantly discoverable, beautifully presented, and technically perfect for search engine algorithms.